

Fundamentos de Aprendizagem Automática

Class 04 — Probabilistic Models

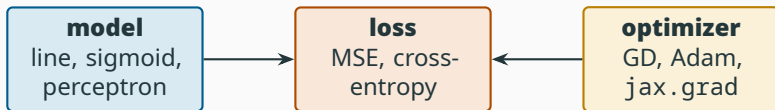
Mário Antunes

Universidade de Aveiro

2026

From Losses to Probabilities

Where We Left Off



Class 04: where do these losses come from?
What if the model outputs a *distribution*?

- Class 03: three models, each a **choice of loss**; ridge and lasso added a *penalty*
- Class 04: the losses are **negative log-likelihoods**, the penalties are **priors**
- A new family, **Naive Bayes**, is trained by *counting*, no optimizer: for classes **and** for numbers
- And a question that accuracy hides: are the **probabilities** right, and what should we *do* with them?

Why These Losses?

Class 03	Loss	Class 04 answer
linear regression	squared error	Gaussian noise (MLE)
robust regression	absolute error	Laplace noise (MLE)
logistic regression	cross-entropy	Bernoulli labels (MLE)
ridge	$\text{MSE} + \ w\ ^2$	Gaussian prior on w (MAP)
lasso	$\text{MSE} + \ w\ _1$	Laplace prior on w (MAP)
Laplace smoothing	add-one counts	Beta prior on a probability (MAP)

- One idea: choose the parameters that make the data **most probable** (MLE), possibly **weighted by a prior** (MAP)

Probability in One Slide

$$P(x, y) = P(x \mid y) P(y), \quad P(x) = \sum_y P(x, y)$$

$$P(y \mid x) = \frac{P(x \mid y) P(y)}{P(x)}$$

Spam data (4601 e-mails)	Value
$P(\text{spam})$ (prior)	0.394
$P(\text{"free" present})$	0.270
$P(\text{"free" \mid spam}), P(\text{"free" \mid ham})$	0.546, 0.090
$P(\text{spam} \mid \text{"free"}) = \frac{0.546 \cdot 0.394}{0.270}$	0.797

- **Conditional independence** given c :

$$P(x_1, x_2 \mid c) = P(x_1 \mid c) P(x_2 \mid c)$$

A Model Is a Distribution

- A probabilistic model gives, for every input, a **distribution** over the target: $p(y \mid x; \theta)$
- Linear regression: $\mathcal{N}(w^\top x + b, \sigma^2)$; logistic: $\text{Bernoulli}(\sigma(w^\top x + b))$
- The **likelihood**: the probability of the *observed* data, as a function of θ :

$$L(\theta) = p(D \mid \theta) = \prod_{i=1}^N p(y_i \mid x_i; \theta)$$

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^N \log p(y_i \mid x_i; \theta)$$

- A **product** of probabilities; its logarithm is a **sum**. $L(\theta)$ is **not** a distribution over θ

Maximum Likelihood

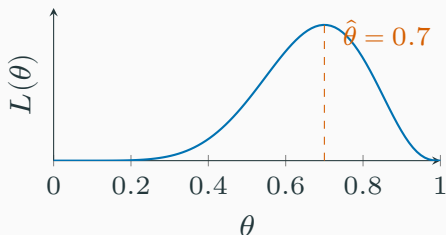
The Maximum Likelihood Estimate

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \ell(\theta) = \arg \min_{\theta} \underbrace{- \sum_i \log p(y_i | x_i; \theta)}_{\text{negative log-likelihood (NLL)}}$$

- Choose the parameters under which the data we saw are **least surprising**
- Maximizing ℓ = minimizing the NLL: **this is the loss** we have been minimizing
- Class 01: $-\log p$ is a **code length** in nats; MLE picks the model that compresses the data best (MDL without the model cost)
- Two ways to solve it: set the derivative to zero (**closed form**, often *counting*) or descend the NLL with `jax.grad`

Worked Example: a Coin

$n = 10$ tosses, $k = 7$ heads. Likelihood of the head probability θ : $L(\theta) = \theta^7(1 - \theta)^3$



$$\ell(\theta) = k \log \theta + (n - k) \log(1 - \theta), \quad \ell'(\theta) = \frac{k}{\theta} - \frac{n - k}{1 - \theta} = 0$$

- The MLE of a probability is the **relative frequency**: counting

The Same Estimate with `jax.grad`

```
def neg_loglik(phi, k, n):                                # phi = logit of theta
    p = jax.nn.sigmoid(phi)
    return -(k * jnp.log(p) + (n - k) * jnp.log1p(-p))

grad = jax.jit(jax.grad(neg_loglik))
phi = 0.0
for _ in range(200):
    phi = phi - 0.1 * grad(phi, 7, 10)                    # gradient descent
# jax.nn.sigmoid(phi) = 0.7000
```

- We write the negative log-likelihood; JAX gives the derivative. Optimizing the **logit** keeps $\theta \in (0, 1)$
- Same method as every model of Class 03: only the *loss* has a new name

Worked Example: a Gaussian

$$x = (2.1, 1.9, 2.6, 2.4, 3.0), \quad x_i \sim \mathcal{N}(\mu, \sigma^2)$$

$$-\ell(\mu, \sigma) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i (x_i - \mu)^2$$

$$\Rightarrow \quad \hat{\mu} = \bar{x}, \quad \hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$$

	Value
$\hat{\mu} = (2.1 + 1.9 + 2.6 + 2.4 + 3.0)/5$	2.400
$\hat{\sigma}_{MLE}^2$ (divide by n): $0.74/5$	0.148
unbiased (divide by $n - 1$): $0.74/4$	0.185

- `jax.grad` on $(\mu, \log \sigma)$ agrees ($4 \cdot 10^{-16}$); the MLE variance is **biased: 0.798** on average for a true 1 ($n = 5$)

The Loss Is a Negative Log-Likelihood

Noise / label model	NLL (up to constants)	Loss
Gaussian	$(y - \hat{y})^2 / 2\sigma^2$	squared error
Laplace	$ y - \hat{y} /b$	absolute error
Bernoulli	$-y \log p - (1 - y) \log(1 - p)$	cross-entropy
Categorical (softmax)	$-\log p_y$	softmax cross-entropy

- Gaussian NLL gives **exactly** the least-squares weights ($2 \cdot 10^{-15}$ on housing); $\hat{\sigma}^2 = \text{training MSE (0.4546)}$
- The mean NLL of a classifier **is** `log_loss` (0.542279 = 0.542279)
- Two outliers pull the Gaussian fit (slope 0.537) more than the Laplace fit (0.523); true slope 0.50

MLE: Counting or Descending

Model	MLE is found by	Cost
Bernoulli, categorical, Naive Bayes	counting frequencies	one pass over the data
Gaussian mean and variance	sample mean, biased variance	one pass
linear regression	normal equation	$O(Nd^2 + d^3)$
logistic regression, neural networks	gradient descent on the NLL	many passes

- When the derivative can be solved in closed form, MLE is **free of any optimizer**: that is why Naive Bayes trains in one pass
- When it cannot, it is the optimization problem of Class 02, with the NLL as the objective

The Weakness of MLE

n	k	$\hat{\theta}_{MLE}$
3	3	1.000
10	7	0.700
100	70	0.700
1000	700	0.700

- Three heads in three tosses: “tails is **impossible**”. A word never seen in a spam e-mail: “this word can **never** appear in spam”
- MLE fits the data it has, with no way to say “I have seen too little”: **overfitting** again (Class 01)
- Remedy: encode what we believed *before* the data as a **prior**, and combine the two: **MAP**

Live Demo M1-M3: Maximum Likelihood

- Notebook `01_mle_map.ipynb`, sections **M1** to **M3**
- The coin: likelihood, log-likelihood, and `jax.grad` descent to 0.7; the Gaussian MLE and its bias
- Gaussian noise $\rightarrow$ MSE, Bernoulli $\rightarrow$ cross-entropy, Laplace $\rightarrow$ MAE with outliers
- Question: which noise model would you choose for house prices with a few luxury mansions?

Maximum A Posteriori

Bayes' Rule for the Parameter

$$p(\theta \mid D) = \frac{p(D \mid \theta) p(\theta)}{p(D)} \propto \underbrace{p(D \mid \theta)}_{\text{likelihood}} \underbrace{p(\theta)}_{\text{prior}}$$

$$\hat{\theta}_{MAP} = \arg \max_{\theta} [\log p(D \mid \theta) + \log p(\theta)] = \arg \min_{\theta} [\text{NLL} +$$

- The **prior** $p(\theta)$: what we believe about the parameter before the data. The **posterior**: after
- MAP = MLE **plus a penalty** — $\log p(\theta)$: the regularization of Class 03, now with a meaning
- As N grows the likelihood (a sum of N terms) overwhelms the prior: **MAP** $\rightarrow$ **MLE**

Beta Prior for a Probability

$$\theta \sim \text{Beta}(a, b) \implies \theta \mid D \sim \text{Beta}(a + k, b + n - k)$$

$$\hat{\theta}_{MAP} = \frac{k + a - 1}{n + a + b - 2} \qquad \mathbb{E}[\theta \mid D] = \frac{k + a}{n + a + b}$$

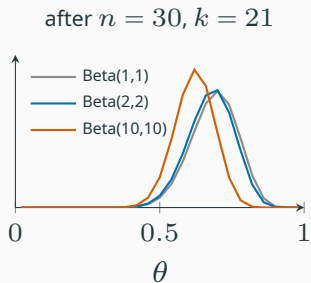
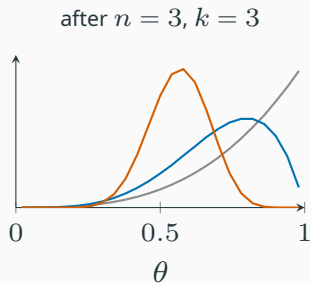
- $a - 1$ and $b - 1$ are **imaginary tosses** seen before the data; $a = b = 1$: none (MAP = MLE)
- $a = b = 2$: one imaginary head and tail:
 $\hat{\theta}_{MAP} = (k + 1)/(n + 2)$, **Laplace's rule of succession**
- The posterior is a full distribution: we know *how sure* we are

Worked Example: the Prior Matters, then Fades

n	k	MLE	MAP	MAP	posterior
			Beta(2,2)	Beta(10,10)	mean Beta(2,2)
3	3	1.000	0.800	0.571	0.714
10	7	0.700	0.667	0.571	0.643
30	21	0.700	0.688	0.625	0.676
300	210	0.700	0.699	0.689	0.697

- Three heads in three tosses: the Laplace prior says **0.8** (not 1), the strong prior 0.571 (almost the prior mean 0.5)
- With 300 tosses all estimates agree: **the data win**. A strong prior needs more data to be overcome

The Posterior Shrinks Around the Data



- Posteriors under three priors: with 3 tosses they disagree (the strong prior sits near 0.58); with 30 they nearly coincide near 0.7
- The width shrinks like $1/\sqrt{n}$: more data, more certainty

Laplace Smoothing Is a Bayesian Prior

- Naive Bayes estimates a word's probability from counts:

$$\theta = \frac{n_{cj}}{n_c} \text{ (MLE). A word never seen in spam gets } \theta = 0$$

$$\theta_{cj} = \frac{n_{cj} + \alpha}{n_c + 2\alpha} = \hat{\theta}_{MAP} \text{ under a Beta}(\alpha+1, \alpha+1) \text{ prior}$$

A word seen in 0 of 8 spam e-mails	θ
MLE ($\alpha = 0$)	0
BernoulliNB(alpha=1) = Beta(2,2)	0.1000
BernoulliNB(alpha=3) = Beta(4,4)	0.2143

- "Add α imaginary e-mails of each kind": an **assumption**, not a trick. It reappears in Naive Bayes (next section) and matches scikit-learn exactly

Ridge Is a Gaussian Prior

Noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, prior $w_j \sim \mathcal{N}(0, \tau^2)$:

$$-\log p(w \mid D) = \frac{1}{2\sigma^2} \sum_i (y_i - w^\top x_i - b)^2 + \frac{1}{2\tau^2} \|w\|^2 + \text{const}$$

$$\implies \text{ridge with } \alpha = \sigma^2 / \tau^2$$

- MAP by `jax.grad` equals
`Ridge(alpha=sigma2/tau2)` to 10^{-15} (here $\alpha = 0.4/0.25 = 1.6$)
- The prior shrinks: weight norm $1.740 \rightarrow 1.468$ (100 housing districts)
- A **Laplace** prior gives the **lasso**; Class 01: the penalty is the **code length of the model**

More Than a Point: the Posterior of the Weights

$$w \mid D \sim \mathcal{N}(\mu, \Sigma), \quad \Sigma = \left(\frac{X^\top X}{\sigma^2} + \frac{I}{\tau^2} \right)^{-1}$$

$$\mu = \frac{\Sigma X^\top y}{\sigma^2} \quad (\text{the ridge weights})$$

Housing, 100 examples	posterior mean	posterior std
MedInc	0.621	0.127
AveOccup	-0.269	0.062
Latitude	-1.078	0.211
Longitude	-1.004	0.200

- Each weight has an **uncertainty**; predictions get one too: $\text{var}(\hat{y}_*) = \sigma^2 + x_*^\top \Sigma x_*$
- The 90% interval covers **0.905** of the test districts. The MAP is only the *peak*; the posterior is the whole answer

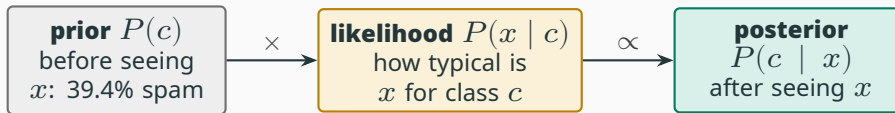
Live Demo M4-M5: MAP

- Notebook `01_mle_map.ipynb`, sections **M4** and **M5**
- Beta posteriors for three priors and four data sizes;
`BernoulliNB(alpha)` = a Beta prior
- Ridge as MAP found by `jax.grad`; the Gaussian posterior of the weights and its intervals
- Question: how would a *lasso* prior change the posterior of the weights?

Naive Bayes

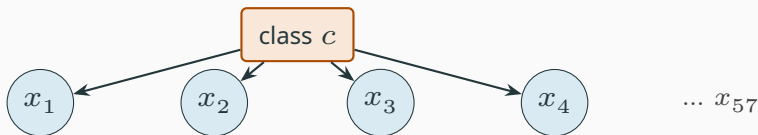
Bayes' Rule for Classification

$$P(c | x) = \frac{P(c) P(x | c)}{P(x)} \propto P(c) P(x | c)$$



- Choose the class with the largest posterior:
 $\hat{c} = \arg \max_c P(c) P(x | c)$. The evidence $P(x)$ is the same for every class, so it can be ignored
- A **generative** model: it models *how each class produces the features*, then inverts with Bayes
- Prior: from class counts (0.394 spam). The hard part is the likelihood $P(x | c)$ of 57 features together

The Naive Assumption



$$P(x_1, \dots, x_d \mid c) = \prod_{j=1}^d P(x_j \mid c)$$

$$\hat{c} = \arg \max_c \left[\log P(c) + \sum_{j=1}^d \log P(x_j \mid c) \right]$$

- Given the class, the features are **independent**: false in practice, yet it works
- A full joint over 57 binary features: $3 \cdot 10^{17}$ numbers; the naive model: **115**

Training Is Counting

Feature	Model of $P(x_j \mid c)$	Estimated by (max. likelihood)
binary (word present)	Bernoulli(θ_{cj})	$\theta_{cj} = n_{cj}/n_c$
categorical (a bin)	categorical	relative frequency of each value in class c
continuous prior	Gaussian(μ_{cj}, σ_{cj}^2) $P(c)$	class mean and class variance n_c/n

- n_c : training examples of class c ; n_{cj} : those in which feature j is present
- **No gradient, no iteration, no learning rate:** one pass over the data
- Prediction: a **sum of logs**, one term per feature. In the lab it is a single matrix operation, scored for all e-mails at once with `jax.numpy`

Worked Example: a Six-E-Mail Corpus

e-mail	free	money	meeting	class
1	1	1	0	spam
2	1	0	0	spam
3	0	1	0	spam
4	0	0	1	ham
5	1	0	1	ham
6	0	1	1	ham

counts n_{cj}	free	money	meeting	n_c
spam	2	2	0	3
ham	1	1	3	3

- Priors: $P(\text{spam}) = P(\text{ham}) = 0.5$
- New e-mail: contains free and meeting, no money. Spam or ham?
- Note the zero: meeting never appeared in a spam e-mail

Worked Example: Without and With Smoothing

$\theta_{cj} = (n_{cj} + \alpha) / (n_c + 2\alpha)$ for a binary feature.

$x = (\text{free} = 1, \text{money} = 0, \text{meeting} = 1)$

	$\alpha = 0$	$\alpha = 1$
$\theta_{\text{spam}} (\text{free}, \text{money}, \text{meeting})$	0.667, 0.667, 0	0.6, 0.6, 0.2
θ_{ham}	0.333, 0.333, 1	0.4, 0.4, 0.8
spam score $P(c)P(x c)$	$0.5 \cdot 0.667 \cdot 0.333 \cdot \mathbf{0} = \mathbf{0}$	$0.5 \cdot 0.6 \cdot 0.4 \cdot 0.2 = 0.024$
ham score	$0.5 \cdot 0.333 \cdot 0.667 \cdot 1 = 0.111$	$0.5 \cdot 0.4 \cdot 0.6 \cdot 0.8 = 0.096$
$P(\text{spam} x)$	0	$0.024 / 0.120 = \mathbf{0.2}$

- Without smoothing a single unseen word **veto**es a class for ever: the posterior is exactly 0
- Laplace smoothing adds α imaginary occurrences of each outcome: rare is not impossible. Verdict: **ham** (posterior 0.8)

Laplace Smoothing on Real Data

Training e-mails	$\alpha = 0$: accuracy	test e-mails with an impossible class	$\alpha = 1$: accuracy
20	0.808	807 of 921	0.836
40	0.841	663	0.860
100	0.849	551	0.885
1000	0.895	93	0.894

- Spam data, Bernoulli NB on “word present”. With $\alpha = 0$ most test e-mails contain a word unseen in one class: that class scores $-\infty$
- The lab agrees: 20 training e-mails, **771 of 921** impossible (accuracy 0.788 against 0.806)
- The effect fades with data, never for **rare** words: $\alpha = 1$ (Laplace) or tuned ($\alpha < 1$: Lidstone)
- It is a **MAP estimate** (previous section): α imaginary observations. Gaussian NB has a variance floor, `var_smoothing`

Work in Log Space

$$\log P(c \mid x) = \log P(c) + \sum_j \log P(x_j \mid c) - \log Z$$

- 57 factors around 0.05 give 10^{-74} ; a text model with 50 000 words underflows to 0.0 in floating point. **Sums of logs never do**
- The class is the arg max of the log score: no normalization needed
- For a probability, normalize with logsumexp ($Z = \sum_{c'} P(c')P(x \mid c')$)
- Bernoulli score with `jnp.where`: $\log \theta_{cj}$ if the word is present, $\log(1 - \theta_{cj})$ if not. This also avoids the $0 \cdot (-\infty)$ that $x \log \theta + (1 - x) \log(1 - \theta)$ produces for an unseen word

What the Model Learned

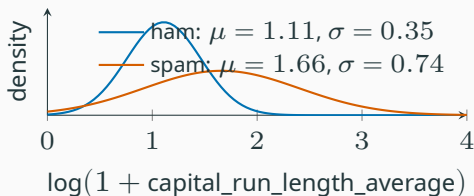
Spam words	log ratio	Ham words	log ratio
remove	3.27	george	-4.06
money	2.96	cs	-3.70
credit	2.60	telnet	-3.58
000	2.49	857	-3.52
addresses	2.18	labs	-2.80
3d	2.01	hpl	-2.75

- Log ratio = $\log P(w \mid \text{spam}) / P(w \mid \text{ham})$: each word adds evidence independently; easy to inspect
- "george", "cs", "hpl": the ham is one person's mailbox at HP Labs, a **dataset artifact** (shortcuts, Class 01)
- Bernoulli NB: test accuracy **0.893**, MCC **0.774**

Continuous Features: Gaussian Naive Bayes

$$P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2}\right)$$

$\hat{\mu}_{cj}$ and $\hat{\sigma}_{cj}^2$: the class mean and class variance of feature j



- One bell per feature and class (2×57 means and variances); spam shouts more, and more variably
- Different variances per class: a **quadratic** boundary (logistic regression: linear)

Categorical Features: More Than Two Values

$$\theta_{cjm} = P(x_j = m \mid c) = \frac{n_{cjm} + \alpha}{n_{cj} + \alpha M} \quad m = 1, \dots, M$$

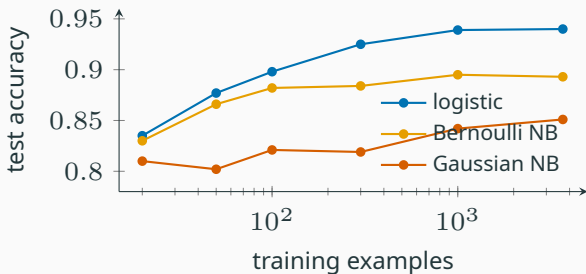
- A feature with M values (a bin of a numeric feature, a colour, a word count class): one probability **per value**, by counting; $M = 2$ is the Bernoulli case
- Numeric features become categorical by **binning**: quantile bins keep every value populated. No shape is assumed for $P(x_j \mid c)$
- α is again a MAP prior (a Dirichlet with α imaginary observations of each value)
- Spam, 4 quantile bins per feature: accuracy **0.866**, MCC **0.719** (many word bins collapse to “0” and “not 0”)

Discrete or Continuous?

Spam test set (921 e-mails)	Accuracy	MCC
Gaussian NB on raw frequencies	0.845	0.715
Gaussian NB on $\log(1 + x)$	0.851	0.725
Categorical NB, 4 quantile bins	0.866	0.719
Bernoulli NB, "word present"	0.893	0.774
logistic regression ($C = 0.1$)	0.937	0.869

- Word frequencies are **not Gaussian** (a spike at 0 and a long tail): the *coarsest* representation (present or not) wins among the NB models
- The logarithm helps the Gaussian a little; **discretizing** makes no distributional assumption, at the price of choosing bins
- The right question is always "what does $P(x_j \mid c)$ really look like?". Look at histograms first

Generative Versus Discriminative



- **NB** models $P(x \mid c)$: many small, easy estimates; converges fast, but plateaus at the error of its independence assumption
- **Logistic regression** models $P(c \mid x)$ directly: fewer assumptions; needs more data, then wins (Ng and Jordan, 2001)
- Close at 20 examples; from about 100 logistic is ahead (0.940 against 0.893 and 0.851). Mean of 10 random subsets

Live Demo N1-N3: Naive Bayes for Classification

- Notebook `02_naive_bayes_classification.ipynb`, sections **N1** to **N3**
- **N1**: the six-e-mail corpus by hand and by code; Bernoulli NB; the smoothing experiment
- **N2**: class-conditional Gaussians, raw versus log features, categorical (binned) features, quadratic versus linear boundary
- **N3**: learning curves of NB and logistic regression
- Question: what happens to Bernoulli NB if we add a duplicate of the word free?

Naive Bayes for Regression

Naive Bayes When the Target Is a Number

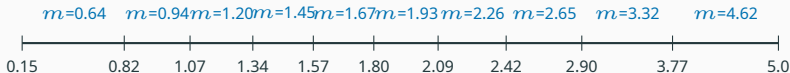
- Naive Bayes needs $P(x_j \mid y)$ and a prior $P(y)$. So far y was a **class**; nothing forces it
- House prices: y is a number. Two routes, each with **discrete** or **continuous** features:

Target	Discrete features	Continuous features
a class (spam)	Bernoulli, categorical NB	Gaussian NB
a number, binned	categorical NB on the bins	Gaussian NB on the bins
a number, continuous	bin the target (route 1)	linear-Gaussian NB

- **Route 1:** turn the regression into a classification of price *bins*, predict the expected price
- **Route 2:** model the features as linear-Gaussian in the price; the posterior of the price is a Gaussian, in closed form

Route 1: Discretize the Target

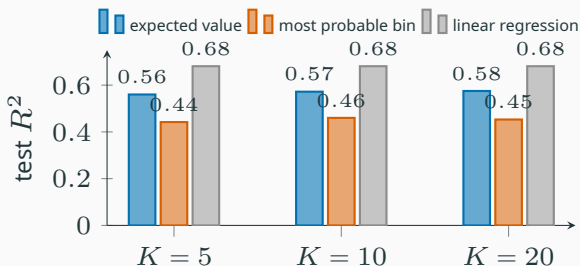
10 quantile bins of the house value (100 000 USD), about 1650 districts each; m_k = bin mean



$$\hat{y}(x) = \sum_{k=1}^K P(k \mid x) m_k \quad (\text{expected value})$$

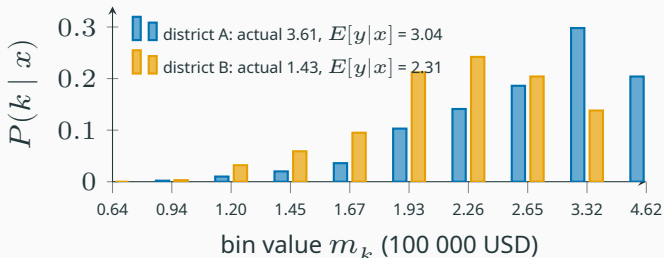
- Each bin is a **class** k with a representative value m_k (the mean of its training targets)
- Any Naive Bayes classifier gives $P(k \mid x)$; the prediction is its **expectation**
- Bins of equal size (quantiles) keep every class populated

Expected Value Versus Most Probable Bin



- Gaussian NB on the standardized housing features; test set of 4128 districts
- The **whole posterior** (the expectation) beats the most probable bin by 0.11–0.12 in R^2 : the argmax can only return one of K values
- More bins help slightly; every NB version stays **below linear regression** (0.681): the features are correlated

A Distribution Over Prices for Every District



- Two test districts: the model is uncertain about both (wide posteriors), and its expectation is **off by 0.6 and 0.9**
- A bonus of the classification route: a **predictive distribution**, not just a number: probability of “more than 300 000 USD”, credible intervals
- Linear regression gives a point only (unless we add a noise model)

Discrete Features for Regression

Test R^2 , $K = 10$ target bins	3 bins	5 bins	10 bins	20 bins
categorical NB on binned features	0.416	0.558	0.625	0.664

- Each feature is cut into quantile bins; CategoricalNB estimates $P(\text{bin of } x_j \mid k)$ by **counting** (with Laplace smoothing)
- Too few bins lose the shape of the feature; with 20 bins the counts approximate each $P(x_j \mid k)$
non-parametrically: 0.664 beats Gaussian NB (0.572) and nears linear regression (0.681)
- The number of bins is a **hyper-parameter**: with more bins each cell holds fewer examples and the smoothing takes over

Route 2: a Continuous Target

$$y \sim \mathcal{N}(\mu_y, \tau^2), \quad x_j \mid y \sim \mathcal{N}(a_j + b_j y, s_j^2)$$

(independent given y)

- MLE: μ_y, τ^2 from the targets; (a_j, b_j) by least squares of **feature on target**; s_j^2 = residual variance
- Every factor is quadratic in y : the posterior is a **Gaussian**

$$P = \frac{1}{\tau^2} + \sum_j \frac{b_j^2}{s_j^2}, \quad \text{sd}(y \mid x) = \frac{1}{\sqrt{P}}$$

$$\hat{y} = \frac{1}{P} \left(\frac{\mu_y}{\tau^2} + \sum_j \frac{b_j (x_j - a_j)}{s_j^2} \right)$$

Worked Example: Two Features

Prior $y \sim \mathcal{N}(2, 1)$; $x_1 \mid y \sim \mathcal{N}(1 + 0.5y, 0.25)$;
 $x_2 \mid y \sim \mathcal{N}(-1 + y, 1)$. Observed $x_1 = 2.4, x_2 = 1.2$.

	precision	vote
prior	$1/\tau^2 = 1$	$\mu_y/\tau^2 = 2$
feature 1	$0.25/0.25 = 1$	$0.5(2.4 - 1)/0.25 = 2.8$
feature 2	$1/1 = 1$	$1(1.2 + 1)/1 = 2.2$

$$P = 3, \quad \hat{y} = \frac{2 + 2.8 + 2.2}{3} = 2.333, \quad \text{sd} = 0.577$$

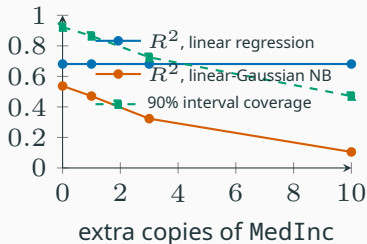
- Each source adds precision 1: the posterior is 3 times more concentrated than the prior
- The votes differ because the observations differ

Result: Linear-Gaussian Naive Bayes on Housing

Feature	b_j (per unit of y)	s_j^2	precision b_j^2/s_j^2
MedInc	0.592	0.532	0.659
AveOccup	-0.218	0.937	0.051
AveRooms	0.205	0.944	0.045
Latitude	-0.122	0.980	0.015
<i>the other four</i>			≤ 0.009

- Test $R^2 = \mathbf{0.537}$, RMSE 0.781 (linear regression 0.681, Gaussian NB on 10 bins 0.572); **median income carries most of the precision**
- Posterior sd **0.807** for *every* district (the precision does not depend on x); the 90% interval covers **0.923**, close to the nominal 0.90
- The formula agrees with a numerical maximum of the log-posterior found by `jax.grad`: $2.842745 = 2.842745$ (district 0)

Where the Naive Assumption Hurts



- Append exact copies of the most informative feature: linear regression is **unchanged** (the copies share one weight)
- Naive Bayes adds the income evidence **again for every copy**: the posterior sd falls from 0.807 to **0.351** while R^2 falls to **0.105**
- With 10 copies the “90%” interval covers **47%**: over-confident *and* wrong

Naive Bayes for Regression: Summary

Model	Target	Features	Posterior	Test R^2
Gaussian NB on 10 bins	binned	continuous	distribution over bins, per district	0.572
categorical NB, 20 bins	binned	discrete	distribution over bins, per district	0.664
linear-Gaussian NB	continuous	continuous	Gaussian, same width everywhere	0.537
linear regression (Class 03)	continuous	continuous	a point	0.681

- Naive Bayes regression is cheap (counting or one least-squares per feature) and gives an **uncertainty**, but the independence assumption makes it **worse than linear regression** when features are correlated
- Use it as a fast baseline, or when a distribution over the target matters more than the last decimal of R^2

Live Demo R1-R4: Naive Bayes for Regression

- Notebook `03_naive_bayes_regression.ipynb`
- **R1:** binned target, expected value versus most probable bin; the posterior of one district
- **R2:** discrete features (3 to 20 bins); **R3:** the continuous target, the closed-form posterior checked with `jax.grad`, interval coverage
- **R4:** duplicate a feature and watch Naive Bayes become confident and wrong

Probabilities You Can Trust

Naive Bayes Is Over-Confident

Model (spam test set)	Accuracy	Log-loss
logistic regression	0.940	0.172
Bernoulli NB	0.893	0.511
Gaussian NB	0.851	3.100

- Same data, three models: Gaussian NB is only 9 points less accurate than logistic regression but its log-loss is **18 times** larger
- It multiplies many correlated factors as if they were independent evidence: its probabilities are pushed to 0 or 1
- **Accuracy cannot see this.** A classifier must be judged as a *probability estimator* too, whenever the probability is used (costs, rankings, alarms)

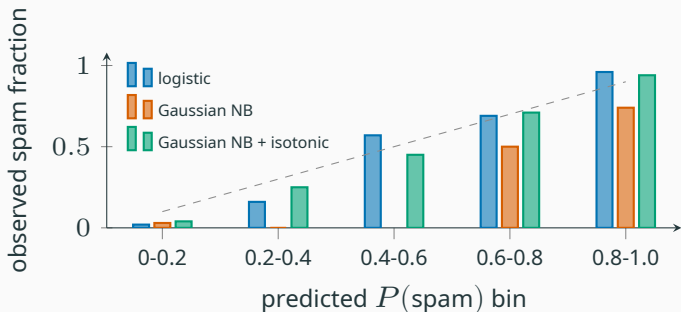
Three Ways to Measure a Probability

$$\text{log-loss} = -\frac{1}{n} \sum_i \log p(y_i | x_i) \quad \text{Brier} = \frac{1}{n} \sum_i (p_i - y_i)^2$$

Spam test set	Accuracy	Log-loss	Brier	ECE
logistic regression	0.940	0.172	0.044	0.026
Bernoulli NB	0.893	0.511	0.091	0.081
Gaussian NB	0.851	3.101	0.148	0.149

- **Log-loss** = mean NLL; unbounded: one confident mistake ($p = 10^{-15}$) costs 34.5 nats
- **Brier**: the MSE of the probability, bounded in $[0, 1]$, more forgiving
- **ECE**: $\sum_b \frac{n_b}{n} |\text{observed}_b - \text{predicted}_b|$ over probability bins; lower is better

Reliability Diagram



- A calibrated model has bars near the bin centres (0.1, 0.3, 0.5, 0.7, 0.9): logistic is close, isotonic-calibrated Gaussian NB is close, **raw Gaussian NB is not**
- Raw Gaussian NB: 472 of 921 e-mails in the top bin, only **74%** spam; 445 in the lowest bin, 3% spam. Almost nothing in between

Calibrating a Classifier

- Keep the classifier; learn a **monotone map** score $\rightarrow$ probability on **held-out** data
- **Platt**: $p = \sigma(as + b)$ (2 parameters). **Isotonic**: any non-decreasing step function, more flexible

Spam test set	Accuracy	Log-loss	Brier	ECE
Gaussian NB	0.851	3.101	0.148	0.149
+ Platt (sigmoid)	0.851	0.361	0.113	0.033
+ isotonic	0.933	0.221	0.058	0.023
Bernoulli NB	0.893	0.511	0.091	0.081
+ Platt	0.893	0.310	0.088	0.038
+ isotonic	0.891	0.285	0.077	0.016

- Gaussian NB log-loss 3.10 $\rightarrow$ 0.22. A monotone map keeps the **ranking**; the isotonic accuracy moves because CalibratedClassifierCV averages 5 fold models. Calibrate on **held-out** data

Platt Scaling with `jax.grad` (Lab F)

- The log-odds of Gaussian NB reach $-6.7 \cdot 10^8$ (median $|s| = 116$): an **over-confidence** symptom. Compress them with $t = \text{sign}(s) \log(1 + |s|)$ (monotone), then fit $\sigma(at + b)$ by the cross-entropy

Lab spam test set	Log-loss	Brier	ECE	Cost (10:1) at t^*
Gaussian NB, raw	3.706	0.147	0.148	1.300
Gaussian NB + Platt	0.312	0.095	0.107	0.390

- Fitted $a = 0.4435$, $b = -1.108$: equal to a LogisticRegression on the same score to $1.3 \cdot 10^{-4}$
- The log-loss falls by a factor 12 and the decision cost by 70%, without touching the classifier

Decisions Under Uncertainty

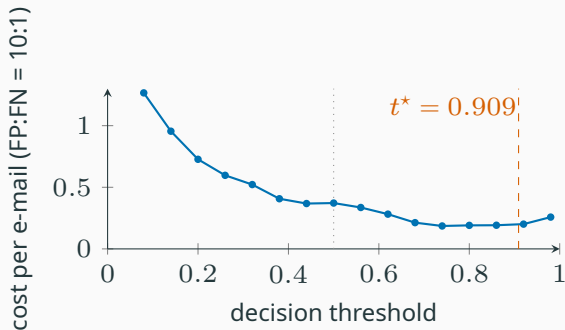
- A false positive (a real e-mail hidden) costs c_{FP} ; a false negative (a spam delivered) costs c_{FN}
- Given $p = P(\text{spam} \mid x)$: flagging costs $(1 - p) c_{FP}$ in expectation, letting it through costs $p c_{FN}$. **Flag when**

$$(1 - p) c_{FP} < p c_{FN} \iff p > t^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$$

Costs FP : FN	t^*
1 : 1	0.500
10 : 1	0.909
1 : 5	0.167

- The threshold is **derived from the costs**, not tuned on the test set, **if p is calibrated**. Class 03's threshold slide gets its theory: 0.5 is right only for equal costs

Worked Example: a Real E-Mail Costs 10 Spam



Logistic, 921 test e-mails	FP	FN	Cost per e-mail
threshold 0.5	32	23	0.372
threshold $t^* = 0.909$	8	99	0.194

- Hiding **24 fewer** real e-mails costs **76 more** delivered spams: worth it at 10:1. The **expected cost halves**, with the same model

Other Costs, Other Thresholds

Costs FP : FN	t^*	Cost at 0.5	Cost at t^*	Best scanned t (cost)
1 : 1	0.500	0.060	0.060	0.40 (0.054)
10 : 1	0.909	0.372	0.194	0.89 (0.180)
1 : 5	0.167	0.160	0.140	0.30 (0.105)

- Cost per e-mail on the test set, logistic regression.
Equal costs: nothing to gain. A costly false negative (1:5) lowers the threshold: flag more
- “Best scanned” uses the test set (cheating): the gap to t^* (0.140 against 0.105 for 1:5) shows how far real probabilities are from perfectly calibrated
- Multi-class and multi-cost problems: pick the action with the **lowest expected cost** under $P(c \mid x)$ (the **Bayes decision**)

Calibrate First, Then Decide

	Cost per e-mail
Lab test set, costs 10:1, threshold $t^* = 0.909$	
logistic regression (calibrated by construction)	0.229
Gaussian NB, raw	1.257
Gaussian NB + Platt scaling	0.390

- A threshold computed from costs is only right for **calibrated** probabilities: the raw Gaussian NB costs **five times more** than logistic regression at the same t^*
- Platt scaling recovers most of the loss ($1.257 \rightarrow 0.390$); the rest is the classifier itself (independence assumption)
- Recipe: **fit** the model $\rightarrow$ **calibrate** on held-out data $\rightarrow$ **decide** with the costs $\rightarrow$ **evaluate** the cost on the test set

Live Demo N4–N5: Calibration and Costs

- Notebook `02_naive_bayes_classification.ipynb`, sections **N4** and **N5**
- Log-loss, Brier and ECE for three models; reliability diagrams; Platt and isotonic scaling with `CalibratedClassifierCV`
- The expected cost against the threshold for three cost ratios; t^* against the best scanned threshold
- Question: what would a *spam filter for a hospital* use as costs?

Summary

Generative Versus Discriminative, Side by Side

	Logistic	Bernoulli NB	Gaussian NB
Models	$P(c x)$	$P(x c)P(c)$	$P(x c)P(c)$
Features	any (scaled)	binary	continuous
Training	GD on the NLL	counting	means, variances
Assumes	linear log-odds	independent	independent, Gaussian
Probabilities	good	over-confident	very over-confident
Spam accuracy	0.940	0.893	0.851

- **Little data:** few parameters, strong assumption (NB) wins; **more data:** logistic wins
- Every model is a **maximum likelihood** or **MAP** estimate

Choosing and Checking

- **A number, a class or a probability?** Regression, classification, or a probability plus a decision
- **Probabilities used for costs, ranking, alarms:** measure log-loss, Brier and reliability, not accuracy; **calibrate**, then decide
- **Fast baseline, little data, discrete features:** Naive Bayes. **Correlated features, more data:** logistic or linear regression
- **Priors:** small data or rare events $\Rightarrow$ a prior (smoothing, ridge) is not optional
- **Watch for:** zero counts, repeated or correlated evidence, thresholds left at 0.5 by habit, calibration fitted on the training data



Key Takeaways

- **MLE:** choose the parameters that make the data most probable; the losses of Class 03 are negative log-likelihoods
- **MAP:** add a prior: Gaussian $\rightarrow$ ridge, Laplace $\rightarrow$ lasso, Beta $\rightarrow$ Laplace smoothing; the prior fades with data
- **Naive Bayes:** Bayes' rule + conditional independence; **counting** for discrete features, means and variances for continuous ones
- **Regression too:** discretize the target and classify (expected value), or model the features as linear-Gaussian in y
- **Over-confidence:** independence counts repeated evidence again; log-loss, Brier and reliability expose it; Platt or isotonic scaling repairs it
- **Decisions:** flag when $p > c_{FP} / (c_{FP} + c_{FN})$, if p is calibrated

Lab 04: Build It in JAX

Part	Task	twin
A	MLE and MAP with <code>jax.grad</code> (Gaussian, Beta, ridge)	Ridge, BernoulliNB
B	Bernoulli NB, discrete features	BernoulliNB
C	Gaussian and categorical NB	GaussianNB, CategoricalNB
D	NB regression: binned and continuous target	GaussianNB, CategoricalNB
E	log-loss, Brier, reliability; costs	<code>log_loss</code> , <code>calibration_curve</code>
F	challenge: Platt scaling	LogisticRegression

- Guide `practice/04_probabilistic_models.pdf`, notebook `lab04_probabilistic_models.ipynb`, solution in `solutions/`
- **Project 1** is due at Class 08: use the protocol of Classes 03 and 04

References (1/2)

- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: chapters 1 and 2 (probability, MLE, Bayesian estimation), 4 (classification)
- D. Barber, *Bayesian Reasoning and Machine Learning*, 2012: Naive Bayes, Bayesian estimation
- G. James, D. Witten, T. Hastie, R. Tibshirani, J. Taylor, *An Introduction to Statistical Learning with Applications in Python*, 2023: Bayes classifier, Naive Bayes
- P. Domingos, M. Pazzani, "On the optimality of the simple Bayesian classifier under zero-one loss", *Machine Learning*, 1997

References (2/2)

- A. Ng, M. Jordan, "On discriminative vs. generative classifiers", *NIPS*, 2001
- J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods", 1999
- B. Zadrozny, C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates", *KDD*, 2002
- A. Niculescu-Mizil, R. Caruana, "Predicting good probabilities with supervised learning", *ICML*, 2005
- C. Elkan, "The foundations of cost-sensitive learning", *IJCAI*, 2001